

CEBU INSTITUTE OF TECHNOLOGY – UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Project Management Plan (SPMP)

for

Smart Document Evaluator (Smart Docs Validator)

Prepared by: Alexandrei Nash Dinapo · Jeffer Azcona · Jushua Peter Te · Ryan Bebiro

Document version: 3.0

Date: May 14, 2026

Note on templates: Structured to follow a typical **SPMP** table of contents (IEEE 1058 / course-style project management). Source Word template `SPMP_TEMPLATE.doc` was not machine-readable; content reflects the **as-built** repository and deployment.

Change History

Version	Date	Description
1.0	May 14, 2026	Initial SPMP for final deliverable package.

Table of Contents

- 1. [Overview](#)
- 2. [Project deliverables](#)
- 3. [Organization and roles](#)
- 4. [Management processes](#)
- 5. [Work breakdown and milestones](#)
- 6. [Schedule and dependencies](#)
- 7. [Risk management](#)
- 8. [Quality assurance](#)
- 9. [Configuration management](#)
- 10. [Deployment and operations](#)
- 11. [References](#)

1. Overview

1.1 Project summary

Smart Document Evaluator (branded **Smart Docs Validator**) is a capstone / course-aligned web system for document submission, AI-assisted rubric evaluation (Google Gemini), teacher review, class roster management, and (as of v3.1) **Course Tasks** with optional handouts and deadline reminders. The production stack is **React + TypeScript + Vite**, **Supabase** (Postgres, Auth, Storage), **Vercel** (static SPA + serverless `api/*`), **Resend** (invitation email), and optional **Google Forms** for usability feedback.

1.2 Objectives

- Deliver a maintainable SPA with secure role separation and RLS-backed data access.

- Provide reliable submission and grading flows with AI + teacher lanes.
- Document requirements (SRS), design (SDD), tests (STD), and management (this SPMP) for academic sign-off.
- Support reproducible deployment (`README.md` , `docs/supabase-*.sql` , environment variables).

1.3 Constraints

- Course timelines and adviser review gates.
- Third-party quotas (Gemini, Resend, Supabase free tiers).
- OAuth-only authentication; no custom password store.

2. Project deliverables

Deliverable	Location / artifact
Source code	GitHub repository (main branch); zip export per course instructions.
SRS	<code>docs/SRS-Smart-Document-Evaluator-v3.md</code> (+ baseline detail in <code>docs/SRS-Smart-Document-Evaluator-v2.md</code>).
SDD	<code>docs/SDD-Smart-Document-Evaluator-v3.md</code> .
SPMP	<code>docs/SPMP-Smart-Document-Evaluator-v3.md</code> (this file).
STD	<code>docs/STD-Smart-Document-Evaluator-v3.md</code> .
ReadMe / deployment	Root <code>README.md</code> (export to PDF separately for submission binder).
Database	<code>npm run db:dump</code> → <code>db-dumps/</code> (gitignored; attach per instructions).
Usability instrument	<code>docs/SoftwareUsabilitySurvey-StudentRateUs.md</code> + generated Google Form.
Research paper	<i>(separate ACM-format document — not stored in this repo by default.)</i>

3. Organization and roles

Role	Responsibility
Team lead / integrator	Git workflow, Vercel/Supabase env alignment, merge readiness.
Frontend	React pages, AuthContext, student/teacher UX, Tailwind styling.
Backend / data	Supabase SQL, RLS policies, Storage policies, <code>api/*</code> serverless.
AI / grading	Gemini payload shaping, heuristic fallback, ReviewQueue integration.
QA	Manual test passes per STD; regression before demos.
Adviser	Scope approval, final presentation go-signal (record sign-off externally).

4. Management processes

4.1 Communication

- Primary: course meetings, adviser consults, team chat (external).
- Technical: GitHub issues/PRs (if used), commit messages, `README.md` updates.

4.2 Change control

- Functional changes require SRS amendment (v3.1 table) + SDD update + STD test updates.
- Schema changes: new file under docs/supabase-*.sql + note in README.md .

4.3 Progress monitoring

- Milestones tied to course calendar (prototype → integration → hardening → documentation → presentation).
- Build health: npm run typecheck , npm run lint , npm run build before merges.

5. Work breakdown and milestones

Phase	Work packages	Exit criteria
Requirements	SRS baseline + v3.1 amendments	Adviser review of scope
Design	SDD context, subsystems, data design	Matches deployed architecture
Implementation	SPA, Supabase, APIs, AI pipeline	Core flows demoable
Hardening	Auth gate, double-submit guards, RLS fixes	No P1 bugs open
Verification	STD execution + usability survey	Evidence in binder
Documentation	SPMP, STD, ReadMe PDF, zip	Checklist complete
Closure	Presentation, peer eval, research paper	Course submission

6. Schedule and dependencies

Dependencies: Supabase project availability; Google Cloud OAuth client; Vercel project; optional Resend domain verification; Gemini API key for live AI demos.

Critical path (typical): Auth + roster gate → submissions + storage → teacher queue → AI integration → polish + docs.

(Insert Gantt or week table here per your course template — placeholder.)

7. Risk management

Risk	Probability	Impact	Mitigation
Gemini outage / quota	Med	High	Heuristic fallback; show notice; record in STD
Resend / DNS failure	Low	Med	Document Gmail CLI fallback; test invite:test
RLS misconfiguration	Med	High	Apply docs/supabase-fix-users-rls-recursion.sql; test student/teacher isolation
PII in committed dumps	Med	High	Keep db-dumps/ gitignored; share dumps out-of-band only
Scope creep	Med	Med	Amend SRS §3.2; adviser approval

8. Quality assurance

- **Static:** TypeScript strict project (npm run typecheck), ESLint.
- **Dynamic:** Manual scenarios in STD; optional Cursor/browser MCP for UI smoke.

- **Usability:** Google Form survey; collate Likert + comments into findings appendix.
 - **Security:** No secrets in client bundle for production Gemini; verify `vercel.json` rewrites.
-

9. Configuration management

- **VCS:** Git; `main` protected for production deploys.
 - **Branches:** Feature branches → PR → merge (team convention).
 - **Secrets:** Vercel + local `.env` (never commit `.env`).
 - **Lockfile:** `package-lock.json` committed for reproducible installs.
-

10. Deployment and operations

- **Frontend:** `npm run build` → `dist/` on Vercel; see `README.md` §Deployment.
 - **Backend:** Supabase SQL editor for migrations; Storage bucket policies.
 - **Monitoring:** Vercel build logs; Supabase Auth logs; browser console for client errors.
 - **Backup:** `npm run db:dump` before major schema changes.
-

11. References

- `docs/SRS-Smart-Document-Evaluator-v3.md`
 - `docs/SDD-Smart-Document-Evaluator-v3.md`
 - `docs/STD-Smart-Document-Evaluator-v3.md`
 - `docs/PRESENTATION-READINESS-AND-DELIVERABLES.md`
 - IEEE Std 1058-1998 (SPMP standard, as reference reading).
-

End of SPMP v3.0